Project 09 — Transfer Learning & Fine-tuning

Level: Intermediate | **Dataset:** Oxford Flowers102 (torchvision) | **Framework:** PyTorch

---

Objective

Fine-tune a pretrained ResNet-50 on a 102-class flower dataset.
Cover: feature extraction vs fine-tuning, layer freezing, differential LR, Optuna HPT, ONNX export.

Project Structure

```
09_transfer_learning/
├── notebooks/
│   ├── 01_data_setup.ipynb
│   ├── 02_feature_extraction.ipynb
│   ├── 03_finetuning.ipynb
│   └── 04_optuna_hpt.ipynb
├── data/
├── models/
│   ├── feature_extract.pkl
│   ├── finetuned.pkl
│   └── best_optuna.pkl
├── charts/
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Notebook 01 — Data Setup (`01_data_setup.ipynb`)

STOP 1 — Load Flowers102

- `torchvision.datasets.Flowers102(download=True)`
- 102 categories, 8189 total images, ~80 per class
- Visualize 3 images per class for 6 classes
- **Agent stops here. Explain:**
  - Why Flowers102 is a good transfer learning benchmark (small dataset, many classes)
  - The fundamental premise of transfer learning: ImageNet features generalize
  - Why building a CNN from scratch on 8189 images would fail (not enough data)
  - What ImageNet is: 1.2M images, 1000 classes — what ResNet was trained on
- Wait for user confirmation before continuing

STOP 2 — ImageNet Normalization

- Always use ImageNet mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]
- Resize to 224×224 (ResNet input requirement)
- Train augmentations: `RandomResizedCrop(224)`, `RandomHorizontalFlip`, `ColorJitter`
- Val/Test: `Resize(256)`, `CenterCrop(224)`
- **Agent stops here. Explain:**
  - Why we MUST use ImageNet normalization with pretrained models
  - What happens if we use different normalization: activations are in wrong range, pretrained features break
  - Why 224×224: ResNet was designed for this input size
  - What RandomResizedCrop does vs CenterCrop
- Wait for confirmation

STOP 3 — DataLoader Setup

- batch_size=32, num_workers=4, pin_memory=True
- Check class balance: Flowers102 is roughly balanced
- **Agent stops here. Explain:**
  - Why balanced datasets are rare in practice
  - What to do when class counts vary in a 102-class problem
- Wait for confirmation

Notebook 02 — Feature Extraction (`02_feature_extraction.ipynb`)

STOP 4 — Load Pretrained ResNet-50

```python
import torchvision.models as models
model = models.resnet50(weights='IMAGENET1K_V1')
print(model)  # understand the architecture
```

- Print total parameters
- **Agent stops here. Explain:**
  - ResNet-50 architecture overview: stem, 4 layer groups, global avg pool, FC
  - What residual connections are: skip connections that allow gradients to flow directly
  - Why ResNet solved the vanishing gradient problem for deep nets (50+ layers)
  - What IMAGENET1K_V1 weights are: trained on 1.2M ImageNet images
  - Total params: ~25M — why we don't want to train all of them on 8k images
- Wait for confirmation

STOP 5 — Freeze All Layers

```python
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(2048, 102)  # replace final FC only
```

- Verify: only `model.fc` params are trainable
- Print trainable vs frozen parameter counts
- **Agent stops here. Explain:**
  - What `requires_grad=False` does: no gradient computed, no weight update
  - Why we freeze: use ResNet as a fixed feature extractor
  - Why only replace `model.fc`: all other layers produce ImageNet features we reuse
  - What 2048 is: ResNet-50's global avg pool output dimension
  - Feature extraction: fast (only FC trains), lower accuracy
- Wait for confirmation

STOP 6 — Feature Extraction Training

- Train ONLY the new FC layer
- 20 epochs, Adam lr=0.001
- Expected accuracy: ~70-75% (fast to converge)
- **Agent stops here. Explain:**
  - Why feature extraction trains so fast (only 102*2048 + 102 = 210k params update)
  - Why accuracy plateaus quickly (backbone frozen, can't adapt to flowers)
  - When feature extraction is enough vs when fine-tuning is needed
- Wait for confirmation

Notebook 03 — Fine-tuning (`03_finetuning.ipynb`)

STOP 7 — Selective Layer Unfreezing

python

```python
# Unfreeze only last 2 ResNet layer groups + FC
for param in model.layer3.parameters():
    param.requires_grad = True
for param in model.layer4.parameters():
    param.requires_grad = True
```

- **Agent stops here. Explain:**
  - Why we don't unfreeze ALL layers: early layers (edges, textures) are universal → don't need updating
  - Why we unfreeze later layers: they encode high-level features that are ImageNet-specific
  - The general rule: the more domain-specific your data, the more layers to unfreeze
  - Risk of unfreezing too many layers on small data: catastrophic forgetting
- Wait for confirmation

STOP 8 — Differential Learning Rates

python

```python
optimizer = Adam([
    {'params': model.layer3.parameters(), 'lr': 1e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(),     'lr': 1e-3},
])
```

- **Agent stops here. Explain:**
  - Why differential LR: pretrained layers need tiny updates (they're already good)
  - New FC needs larger LR (random initialization, needs to train fast)
  - The 10× rule: each layer group gets ~10× smaller LR than the one above it
  - This is the same trick used in ULMFiT (NLP) and discriminative fine-tuning
- Wait for confirmation

STOP 9 — Fine-tuning Training Loop

- 30 epochs, CosineAnnealingLR
- Compare accuracy: feature extraction vs fine-tuning
- Expected improvement: ~5-10% accuracy gain from fine-tuning
- **Agent stops here. Explain:**
  - Why fine-tuning outperforms feature extraction: domain adaptation
  - The risk of fine-tuning without careful LR: destroying pretrained knowledge
  - What "catastrophic forgetting" looks like: accuracy spikes then crashes
  - How CosineAnnealing helps: prevents large late-epoch gradient updates on pretrained layers
- Wait for confirmation

Notebook 04 — Optuna HPT (`04_optuna_hpt.ipynb`)

STOP 10 — Optuna Setup

```python
import optuna

def objective(trial):
    lr_fc = trial.suggest_float('lr_fc', 1e-4, 1e-2, log=True)
    lr_backbone = trial.suggest_float('lr_backbone', 1e-6, 1e-4, log=True)
    dropout = trial.suggest_float('dropout', 0.2, 0.6)
    unfreeze_from = trial.suggest_categorical('unfreeze_from', ['layer3', 'layer4'])
    # build model, train 5 epochs, return val accuracy
    ...
    return val_accuracy

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)
```

- **Agent stops here. Explain:**
  - What Optuna is: Bayesian hyperparameter optimization framework
  - How `suggest_float(log=True)` works: searches in log space (better for LR)
  - What `n_trials=20` means: 20 different hyperparameter combinations tried
  - What direction='maximize' means: we want highest val accuracy
  - Optuna vs grid search: smarter sampling (TPE algorithm), needs fewer trials
- Wait for confirmation

STOP 11 — Optuna Visualization

- `optuna.visualization.plot_param_importances(study)`
- `optuna.visualization.plot_optimization_history(study)`
- Print best trial params
- **Agent stops here. Explain:**
  - How to read param importance: which HP matters most
  - What optimization history shows: does accuracy improve with more trials?
  - When to stop HPT: diminishing returns after ~20 trials for 4 params
- Wait for confirmation

STOP 12 — Final Model with Best Params

- Retrain with best Optuna params, full 30 epochs
- Compare: feature extraction → fine-tuning → HPT fine-tuning accuracies
- ONNX export of final model
- **Agent stops here. Explain:**
  - The three-stage accuracy improvement and why each stage helps
  - Why ONNX export is important for production (as covered in Project 05)
  - Dynamic axes for variable batch size in ONNX
- Wait for confirmation

STOP 13 — Confusion Matrix on 102 Classes

- Compute test accuracy, per-class recall
- Plot top-10 most confused class pairs
- **Agent stops here. Explain:**
  - Why 102-class confusion matrix is unreadable as a full heatmap
  - How to extract most confused pairs: find off-diagonal cells with highest values
  - What visually similar flowers (e.g. roses, peonies) being confused means
  - How to improve: more data, larger model, specific augmentations
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model()` → model, class_names
- `predict_flower(pil_image)` → top-3 class predictions with confidence
- `get_training_comparison()` → 3 accuracy curves (feature extract, finetune, best HPT)
- `get_top_confused_pairs()` → list of (class_a, class_b, count)

`app.py` — Streamlit (~80 lines)

Sections:

1. Upload flower image
2. Show top-3 predictions with confidence bars
3. Tab 1: Training comparison (3 curves on one chart)
4. Tab 2: Top confused class pairs
5. Tab 3: Optuna best params summary

Key Concepts Covered

- ResNet-50 architecture and residual connections
- Feature extraction vs fine-tuning
- Layer freezing with requires_grad=False
- Differential learning rates per layer group
- Catastrophic forgetting prevention
- Optuna Bayesian HPT (suggest_float log=True, n_trials)
- 102-class confusion matrix analysis
- Full transfer learning workflow